

# ADVANCED TEXT GENERATION & SUMMARIZATION

- MODULE 06:  
CONTROLLING PROBABILITY  
& EFFICIENT FINE-TUNING  
(PEFT/LORA)
  - THEME: "TAMING THE  
STOCHASTIC PARROT:  
CONTROL & EFFICIENCY IN  
GENERATION"
- 

## THE AUTOREGRESSIVE ENGINE

---

**Recall:** Language Models predict the \*next token\* based on probability.

---

**The Formula:**  $P(w_t \mid w_{\{1:t-1\}})$

---

**The Dilemma:** The model assigns a probability to \*every word in the dictionary\*.

---

**The Question:** How do we choose?  
Do we always pick the most likely word? (Hint: No).

# STRATEGY 1: GREEDY SEARCH & BEAM SEARCH

**Greedy Search: Always pick the #1 most likely word.**

- Pros: Fast.
- Cons: Repetitive, boring, gets stuck in loops.

**Beam Search: Keep the top N sequences alive at each step, choose best total path at end.**

- Visual: A tree diagram expanding and pruning branches.
- Use Case: Translation & Summarization (accuracy > creativity).

# STRATEGY 2: SAMPLING (THE "CHAOS" KNOBS)

## Random Sampling:

Pick the next  
word based  
on probability  
distribution.

**Temperature  
(T):** Reshapes  
the  
probability  
curve.

- **Low T ( $< 1.0$ ):** Exaggerates peaks.  
Conservative & confident.  
(Coding/Math).
- **High T ( $> 1.0$ ):** Flattens peaks.  
Allows rare words.  
(Poetry/Brainstorming).

# STRATEGY 3: TOP-K & NUCLEUS (TOP-P) SAMPLING

- **THE MODERN STANDARD.**
- **TOP-K:** ONLY SAMPLE FROM THE TOP K (E.G., 50) WORDS. CUTS OFF THE "LONG TAIL".
- **NUCLEUS (TOP-P):** DYNAMIC. SAMPLE FROM SMALLEST SET OF WORDS WHERE CUMULATIVE PROB = P (E.G., 0.9).
  - WHY IT WINS: IT ADAPTS. SET IS SMALL IF MODEL IS SURE, LARGE IF UNSURE.

# ROBOTICS USE CASE: GENERATING MISSION REPORTS

- **SCENARIO:** A ROBOT EXPLORES A CAVE AND SENDS A TEXT UPDATE.
- **CONSTRAINT:** BANDWIDTH IS LOW. WE NEED CONCISE SUMMARIES.
- **STRATEGY:**
  - RAW LOG: [TIME: 10:00, LOC: X,Y, BAT: 80%, OBJ: ROCK...]
  - GENERATED SUMMARY: "EXPLORED SECTOR 4. FOUND GEOLOGICAL OBSTACLES. BATTERY STABLE."

# EXTRACTIVE VS. ABSTRACTIVE SUMMARIZATION

- **EXTRACTIVE (THE HIGHLIGHTER):** SELECTS EXISTING SENTENCES.
  - PROS: FACTUAL. CONS: DISJOINTED FLOW.
- **ABSTRACTIVE (THE GHOSTWRITER):** GENERATES NEW SENTENCES (SEQ2SEQ).
  - PROS: NATURAL FLOW. CONS: PRONE TO HALLUCINATION.

# THE ARCHITECTURE FOR SUMMARIZATION

- **ENCODER-DECODER (T5 / BART / PEGASUS):**
  - ENCODER: READS (UNDERSTANDING).
  - DECODER: WRITES (GENERATION).
- **DECODER-ONLY (GPT-4 / LLAMA 3):**
  - CAN SUMMARIZE VIA "FEW-SHOT PROMPTING", BUT ENCODER-DECODER IS OFTEN MORE EFFICIENT FOR PURE TEXT-TO-TEXT.



# NLP & LLM EVALUATION METRICS CHEATSHEET

Comprehensive Guide to Assessing Language Models & Text Generation



## TEXT GENERATION QUALITY & FLUENCY



### Perplexity

Measures how well a model predicts a sample. Lower perplexity = better performance.



Good For: Language Modeling.  
Speed: Fast.



### BLEU

Compares n-gram overlap between generated and reference text. Scores 0-1 (Higher is better).



Good For: Machine Translation.  
Speed: Very Fast.



### ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Evaluates summarization via n-gram/sequence overlap. ROUGE-N, ROUGE-L variants.



Good For: Summarization.  
Speed: Fast.



### METEOR

Improves BLEU with synonyms, stemming, and alignment. Better human correlation.

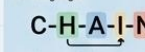


Good For: Translation, Generation.  
Speed: Medium.



### CHRF / CHRF++

Character n-gram F-score. Effective for morphologically rich languages.



Good For: Morphologically Rich Text.  
Speed: Fast.



## QUESTION ANSWERING & ACCURACY



### Exact Match (EM)

Percentage of predictions that exactly match ground truth.



Good For: QA (SQuAD).  
Speed: Very Fast.



### F1 Score (Token-level)

Harmonic mean of precision and recall at token level for partial credit.

$$F_1 = 2 \cdot \frac{(P \cdot R)}{(P + R)}$$

Good For: QA, NER.  
Speed: Fast.



### Factuality / Factual Consistency

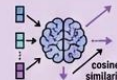
Assesses if outputs align with factual knowledge. Uses external tools.



Good For: Verification, Safety.  
Speed: Slow (Depends on tools).

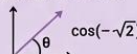


## SEMANTIC & LEARNED METRICS



### BERTScore

Uses BERT contextual embeddings for similarity. Computes P, R, F1 based on cosine.

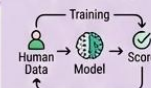


Good For: Semantic Similarity.  
Speed: Slow (Requires BERT).



### BLEURT

(Blue Unsupervised Runtime Testing)  
Learned evaluation metric trained on human judgments. Robust for fluency and quality.



Good For: Fluency, Quality.  
Speed: Medium-Slow.



### MAUVE

Compares divergence between model and human text distributions. Assesses quality & diversity.



Good For: Quality & Diversity.  
Speed: Medium.

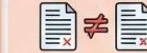


## DIVERSITY, SAFETY & HUMAN EVALUATION



### Self-BLEU

Measures diversity by computing BLEU against itself. Lower score = Higher diversity.

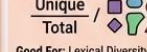


Good For: Diversity.  
Speed: Fast.



### Distinct-n

Unique n-grams divided by total n-grams. Evaluates lexical diversity.



Good For: Lexical Diversity.  
Speed: Fast.



### Toxicity Score

Measures harmful or offensive language using classifiers.



Good For: Safety.  
Speed: Medium (Depends on API).



### Human Evaluation

Gold standard. Raters assess fluency, coherence, relevance, and factuality.

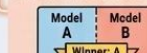


Good For: Validation, Final Check.  
Speed: Very Slow & Costly.



### Win Rate (Pairwise Comparison)

A/B testing; measures how often one model is preferred over another.



Good For: Comparison, Selection.  
Speed: Medium-Slow (Requires judging).

**SUMMARY:** Combine multiple metrics for a holistic assessment. No single metric captures all quality aspects. Balance automatic evaluation with human judgment for reliable results.



# EVALUATION: THE ROUGE METRIC

- **WHY ACCURACY DOESN'T WORK:** NO SINGLE "CORRECT" SUMMARY.
- **ROUGE:** MEASURES OVERLAP OF N-GRAMS BETWEEN GENERATED SUMMARY AND HUMAN REFERENCE.
  - **ROUGE-1:** OVERLAP OF SINGLE WORDS.
  - **ROUGE-L:** LONGEST COMMON SUBSEQUENCE (STRUCTURE).

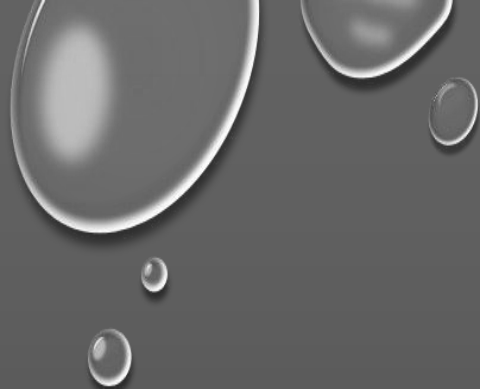
# THE HALLUCINATION DANGER

- **EXAMPLE:**

- SOURCE: "THE ROBOT BATTERY IS AT 10%."
- BAD SUMMARY: "THE ROBOT IS FULLY CHARGED."

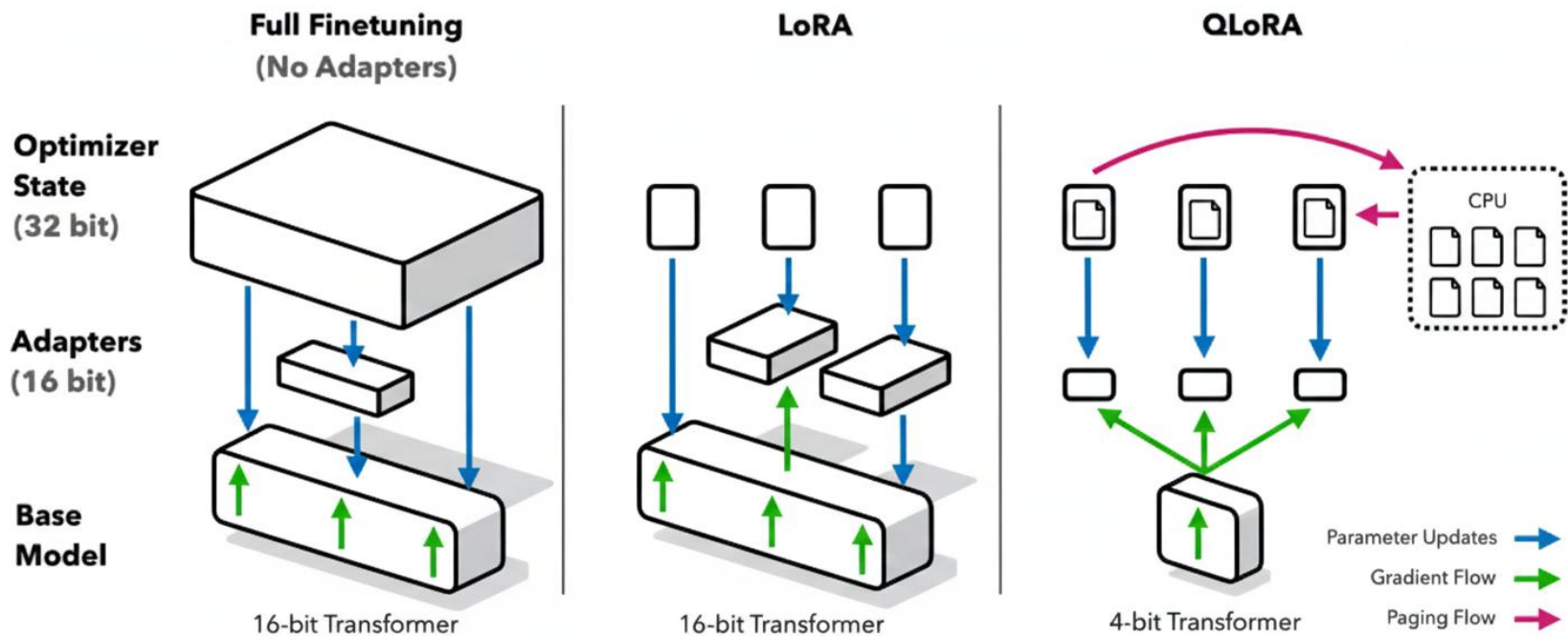
- **MITIGATION:**

- FACT-CHECKING LAYERS (NATURAL LANGUAGE INFERENCE).
- LOW TEMPERATURE (REDUCING CREATIVITY).



# THE FINE- TUNING PROBLEM

- **CONTEXT:** CUSTOMIZING LLAMA-3-8B TO SPEAK LIKE A PIRATE.
- **FULL FINE-TUNING:**
  - REQUIRES UPDATING ALL 8B WEIGHTS.
  - REQUIRES ~120GB+ VRAM (DO YOU HAVE 4 A100 GPUS?).
- **THE SOLUTION:** PEFT (PARAMETER-EFFICIENT FINE-TUNING).



**Figure 1:** Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

# ENTER LORA (LOW-RANK ADAPTATION)

$$\overbrace{W_{\text{ft}}}^{\text{Finetuned Weights}} = \underbrace{W_{\text{pt}}}_{\text{Pretrained Weights}} + \overbrace{\Delta W}^{\text{Weight Update}}$$

- **THE INSIGHT:** WE DON'T NEED TO CHANGE EVERY WEIGHT. THE REQUIRED CHANGE IS LOW-RANK.
- **HOW IT WORKS:**
  - FREEZE THE MASSIVE PRE-TRAINED MODEL ( $W$ ).
  - INJECT TWO TINY MATRICES (A AND B) NEXT TO LAYERS. TRAIN ONLY A AND B.
- **MATH:**  $W_{\text{NEW}} = W_{\text{FROZEN}} + (B \times A)$

$$W_{\text{ft}} = W_{\text{pt}} + \underbrace{\Delta W}_{\text{Approximation}} = W_{\text{pt}} + \underbrace{AB}_{\substack{\text{Rank Decomposition} \\ \text{Matrix}}}$$

where  $W_{\text{ft}}, W_{\text{pt}}, \Delta W, AB \in \mathbb{R}^{d \times d}$   
and  $\underbrace{A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}}_{\text{Low Rank}}$



# THE EFFICIENCY GAINS

- **STORAGE:** FULL LLAMA-3 CHECKPOINT = 15GB.  
LORA ADAPTER = 20MB.
- **MEMORY:** TRAIN A 7B MODEL ON A SINGLE CONSUMER GPU (RTX 4090 / COLAB).
- **MODULARITY:** SWAP ADAPTERS INSTANTLY.
  - ADAPTER 1: MEDICAL DOCS. ADAPTER 2: LEGAL DOCS.
  - BASE MODEL: STAYS THE SAME.

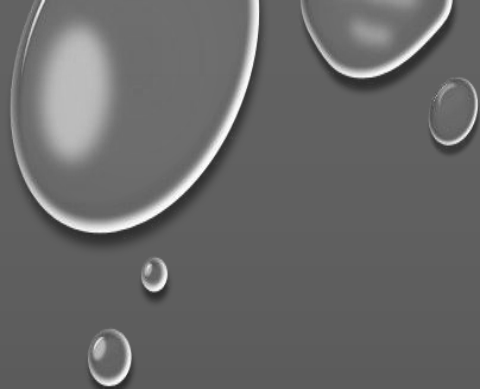




# QLORA (QUANTIZED LORA)

- **2025 STANDARD:**
  - QUANTIZATION: LOAD BASE MODEL IN 4-BIT TO SAVE MEMORY.
  - LORA: APPLY THE ADAPTER ON TOP.
- **RESULT:** FINE-TUNING SOTA MODELS ON CONSUMER HARDWARE.





# ADAPTERS IN ROBOTICS

- **THE "SKILL STORE" CONCEPT:**
  - ROBOT BASE BRAIN: GENERAL LANGUAGE UNDERSTANDING.
  - LORA MODULE A: NAVIGATION SYNTAX.
  - LORA MODULE B: TOOL MANIPULATION MANUAL.
- \*THE ROBOT LOADS THE SPECIFIC LORA ADAPTER BASED ON THE TASK AT HAND.\*

# LAB OVERVIEW

- **PART 1: DECODING PLAYGROUND**

- USE MODEL.GENERATE() WITH DIFFERENT SETTINGS (TEMP 0.1 VS 2.0).
- OBSERVE HOW BEAM SEARCH FIXES GRAMMAR ERRORS.
- EVALUATE USING CHEAT SHEET.

- **PART 2: LORA FINE-TUNING**

- TASK: FINE-TUNE BASE MODEL (FLAN-T5/TINYLLAMA) TO SUMMARIZE DIALOGUES.
- TOOL: HUGGING FACE PEFT LIBRARY.

# SUMMARY

- **CONTROL:** USE NUCLEUS SAMPLING FOR CREATIVITY, BEAM SEARCH FOR ACCURACY.
- **EFFICIENCY:** NEVER FULL FINE-TUNE. USE LORA/PEFT.
- **FUTURE:** ADAPTER FUSION (COMBINING SKILLS).

# REFERENCES & READING

- HOLTZMAN ET AL. (2019). "THE CURIOUS CASE OF NEURAL TEXT DEGENERATION" (NUCLEUS SAMPLING). [HTTPS://ARXIV.ORG/ABS/1904.09751](https://arxiv.org/abs/1904.09751)
- HU ET AL. (2021). "[LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS](#)".
- MIT PAPER ON [CHAIN OF DENSITY PROMPTING](#) (2023).
- [HUGGING FACE PEFT INDEX](#)
- **NEXT WEEK:** MODULE 7 - RAG SYSTEMS  
RAG SURVEY (2023) - [HTTPS://ARXIV.ORG/PDF/2312.10997](https://arxiv.org/pdf/2312.10997)